LevelUp Cloud Trainings
levelupedu.net | Hyderabad, India

GRAND ASSIGNMENT

Apache Spark – RDD

Grand Assignment

Transformations · Actions · Real-World Industry Scenarios

25 Questions | Multi-Industry | Sample Datasets Included | PySpark / Databricks

Trainer	Total Marks	Difficulty	Platform
Sateesh (MCT)	100	Easy / Medium / Hard	Databricks / PySpark

Sateesh Says:
Think of RDDs like a Mumbai Dabba system — each dabba (partition) travels independently but together they deliver the full meal. Transformations prep the food (LAZY); Actions serve it (EAGER)!

SECTION 1: RDD Transformations

eCommerce | Easy

Q1 Flipkart stores product listings as 'ProductName,Category'. Separate product names from categories and print only the distinct categories available.

RDD Operations: `map()` | `distinct()` | `collect()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "iPhone 15,Electronics",
  "Samsung Galaxy,Electronics",
  "Nike Shoes,Footwear",
  "Levi Jeans,Apparel",
  "Sony Headphones,Electronics",
  "Puma Sneakers,Footwear",
  "HP Laptop,Electronics",
  "Adidas T-Shirt,Apparel",
  "Boat Earbuds,Electronics",
  "Woodland Boots,Footwear"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Apply map() to split each line on comma and extract only the Category field
- Use distinct() to eliminate duplicate category names
- Collect and print all unique categories

Healthcare | Easy

Q2 Apollo Hospitals records are stored as 'PatientID,Disease,Age'. Filter out all patients under 18 and return only adult patient records with their disease information.

RDD Operations: `map()` | `filter()` | `count()` | `take()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "P001,Diabetes,45",
  "P002,Asthma,16",
  "P003,Hypertension,52",
  "P004,Fever,12",
  "P005,Typhoid,28",
  "P006,Dengue,9",
  "P007,Cancer,67",
  "P008,Malaria,34",
  "P009,Chickenpox,15",
  "P010,Arthritis,71"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Parse each record by splitting on comma — convert Age to integer
- Use filter() to keep only patients where Age >= 18
- Count adult patients and display first 5 records using take(5)

Pharma | Easy

Q3 Dr. Reddy's Laboratories stores drug synonyms as semicolon-separated strings (e.g., 'Aspirin;Ecosprin;Disprin'). Create a flat, sorted, deduplicated list of all individual drug names.

RDD Operations: `flatMap()` | `distinct()` | `sortBy()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "Aspirin;Ecosprin;Disprin",
  "Paracetamol;Crocin;Dolo650",
  "Metformin;Glucophage;Glycomet",
  "Aspirin;Loprin",
  "Atorvastatin;Lipitor;Storvas",
  "Paracetamol;Tylenol",
  "Amoxicillin;Amoxil;Trimox",
  "Ibuprofen;Brufen;Advil"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Use flatMap() to split each record on semicolon and explode into individual names
- Remove duplicates using distinct()
- Sort alphabetically using sortBy() and collect the final list

Banking | Medium

Q4 HDFC Bank has two separate customer lists — premium account holders and loan defaulters. Identify: (a) safe premium customers who are NOT defaulters, (b) risky customers who appear in BOTH lists, and (c) a combined unique list of all.

RDD Operations: `subtract()` | `intersection()` | `union()` | `count()`

```
# Sample Dataset  (use sc.parallelize() to load)

premium_customers = [
  "CUST001","CUST002","CUST003","CUST004","CUST005",
  "CUST006","CUST007","CUST008","CUST009","CUST010"
]
defaulters = [
  "CUST003","CUST007","CUST011","CUST012","CUST013",
  "CUST002","CUST015","CUST016"
]
premium_rdd = sc.parallelize(premium_customers)
defaulters_rdd = sc.parallelize(defaulters)
```

Tasks to Perform:

- Use subtract() to get premium customers who are NOT defaulters (safe list)
- Use intersection() to find customers common to both lists (risky accounts)
- Use union() followed by distinct() to build a complete unique customer universe
- Count each resulting RDD and print results

Telecom | Medium

Q5 Jio CDR (Call Detail Records) are stored as 'CallerID,ReceiverID,DurationSecs,City'. Group all calls by CallerID and calculate total call duration per caller. Find the top 5 highest-usage callers.

RDD Operations: `map()` | `reduceByKey()` | `sortBy()` | `take()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "JIO001,JIO005,240,Mumbai",
  "JIO002,JIO008,180,Delhi",
  "JIO001,JIO003,320,Mumbai",
  "JIO003,JIO001,90,Pune",
  "JIO002,JIO004,450,Delhi",
  "JIO004,JIO006,210,Chennai",
  "JIO001,JIO007,500,Mumbai",
  "JIO005,JIO002,130,Hyderabad",
  "JIO003,JIO004,370,Pune",
  "JIO002,JIO001,280,Delhi"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Parse each record and create (CallerID, DurationSecs) key-value pairs
- Use reduceByKey(lambda a,b: a+b) to total call duration per caller
- Sort by duration descending using sortBy(lambda x: -x[1])
- Use take(5) to get the top 5 callers

Logistics | Medium

Q6 Delhivery manages parcels with data format 'TrackingID,Status,City,WeightKg'. Group parcels by City and calculate total weight being handled per city. Also find cities handling more than 50 kg.

RDD Operations: `map()` | `groupByKey()` | `mapValues()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "TRK001,InTransit,Mumbai,12.5",
  "TRK002,Delivered,Delhi,8.0",
  "TRK003,InTransit,Mumbai,22.0",
  "TRK004,Pending,Hyderabad,5.5",
  "TRK005,InTransit,Delhi,18.0",
  "TRK006,Delivered,Bangalore,9.0",
  "TRK007,InTransit,Mumbai,30.0",
  "TRK008,InTransit,Delhi,25.0",
  "TRK009,Pending,Hyderabad,14.0",
  "TRK010,InTransit,Bangalore,17.5"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Create (City, WeightKg) pairs using map() — convert weight to float
- Group weights per city using groupByKey()
- Calculate total weight per city using mapValues(lambda vals: sum(vals))
- Filter cities with total weight > 50 kg

Retail | Easy

Q7 Big Bazaar's sales data format is 'ItemName,Quantity,PriceRs'. Add a computed column 'TotalValue = Quantity * Price' for each item and filter out items with TotalValue below Rs. 500.

RDD Operations: `map()` | `filter()` | `collect()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "Rice,10,45.0",
  "Cooking Oil,2,180.0",
  "Sugar,5,42.0",
  "Biscuits,8,30.0",
  "Milk,20,60.0",
  "Shampoo,3,220.0",
  "Detergent,4,95.0",
  "Chips,15,20.0",
  "Soap,12,40.0",
  "Toothpaste,6,85.0"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Parse each line; compute TotalValue = int(Quantity) * float(Price)
- Use map() to produce a tuple (ItemName, Quantity, Price, TotalValue)
- Filter items where TotalValue >= 500 using filter()

- Collect and print the final enriched records

Education | Hard

Q8 BYJU's student score data format is 'StudentID,Subject,Score'. Find average score per subject, identify the top scorer per subject, and flag students below 40 marks as 'At Risk'.

RDD Operations: `groupByKey()` | `mapValues()` | `flatMap()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "S001,Maths,78",   "S002,Maths,45",
  "S003,Maths,92",   "S004,Maths,35",
  "S005,Science,88", "S006,Science,38",
  "S007,Science,67", "S008,Science,72",
  "S001,English,56", "S002,English,82",
  "S003,English,29", "S004,English,91",
  "S005,History,44", "S006,History,63",
  "S007,History,37", "S008,History,55"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Parse records into (Subject, (StudentID, Score)) pairs
- Use groupByKey() to collect all scores per subject
- Compute average score per subject using mapValues()
- Create a second RDD of (StudentID, Subject, 'At Risk') for scores < 40
- Find the top scorer per subject using max() inside mapValues()

Manufacturing | Medium

Q9 Tata Motors IoT sensor data format is 'SensorID,TemperatureC,PressureBar,Timestamp'. Flag sensors in critical condition (Temp > 90C OR Pressure > 150 bar) and count alerts per SensorID.

RDD Operations: `filter()` | `map()` | `countByKey()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "S01,85,140,2024-01-15 09:00",
  "S02,95,145,2024-01-15 09:01",
  "S01,92,155,2024-01-15 09:02",
  "S03,78,130,2024-01-15 09:03",
  "S02,88,152,2024-01-15 09:04",
  "S04,102,148,2024-01-15 09:05",
  "S03,91,135,2024-01-15 09:06",
  "S01,76,160,2024-01-15 09:07",
  "S04,80,145,2024-01-15 09:08",
  "S02,110,158,2024-01-15 09:09"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Parse each record; convert Temperature and Pressure to float
- Apply filter() with OR condition: Temp > 90 OR Pressure > 150
- Map each critical alert to (SensorID, 1) pairs
- Use countByKey() to get total critical alerts per sensor — print sorted by count

Media/OTT | Hard

Q10 **Hotstar viewing data format is 'UserID,ContentID,WatchMinutes,Genre'. Find top 3 genres by total watch time AND identify binge-watchers (users with total watch > 200 minutes in the dataset).**

RDD Operations: `map()` | `reduceByKey()` | `top()` | `filter()` | `sortBy()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "U01,C001,45,Action",  "U02,C002,90,Drama",
  "U01,C003,120,Action", "U03,C001,60,Action",
  "U02,C004,80,Comedy",  "U04,C002,110,Drama",
  "U01,C005,75,Comedy",  "U05,C003,95,Action",
  "U03,C004,150,Comedy", "U02,C005,55,Action",
  "U04,C001,40,Action",  "U06,C002,130,Drama",
  "U05,C004,85,Comedy",  "U06,C003,70,Action",
  "U03,C005,100,Drama",  "U07,C001,200,Drama"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Create (Genre, WatchMinutes) pairs and aggregate using reduceByKey()
- Sort genres by total watch time using sortBy() — extract top 3 using take(3)
- Create (UserID, WatchMinutes) pairs and reduce to total per user
- Filter users with total > 200 minutes — these are binge-watchers
- Count binge-watchers and print their UserIDs

SECTION 2: Key-Value Pair RDD Operations

eCommerce | Medium

Q11 **Amazon India has two datasets: Orders ('OrderID,CustomerID,Amount') and Customers ('CustomerID,Name,City'). Join them to produce a full order report with customer names and cities.**

RDD Operations: `map()` | `join()` | `leftOuterJoin()`

```
# Sample Dataset  (use sc.parallelize() to load)

orders_data = [
  "ORD001,C01,1500", "ORD002,C02,3200",
  "ORD003,C01,850",  "ORD004,C03,4700",
  "ORD005,C04,600",  "ORD006,C02,2100",
  "ORD007,C05,9800", "ORD008,C06,350"
]
customers_data = [
```

```
    "C01,Rahul Sharma,Mumbai",
    "C02,Priya Verma,Delhi",
    "C03,Amit Patel,Ahmedabad",
    "C04,Sneha Iyer,Chennai",
    "C05,Ravi Kumar,Hyderabad"
    # Note: C06 is missing — test leftOuterJoin behavior
]
orders_rdd = sc.parallelize(orders_data)
customers_rdd = sc.parallelize(customers_data)
```

Tasks to Perform:

- Convert orders_rdd to (CustomerID, (OrderID, Amount)) pairs
- Convert customers_rdd to (CustomerID, (Name, City)) pairs
- Use join() — only matched records; compare output row count
- Use leftOuterJoin() — all orders even if customer C06 is missing
- Print a formatted report: OrderID | CustomerName | City | Amount

Banking | Hard

Q12 SBI Bank transactions: 'TxnID,AccountNo,Type,AmountRs' where Type is CREDIT or DEBIT. For each account calculate net balance (CREDIT - DEBIT). Flag accounts with negative net balance as 'Overdrawn'.

RDD Operations: `map()` | `reduceByKey()` | `mapValues()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "T001,ACC101,CREDIT,50000",
  "T002,ACC102,CREDIT,30000",
  "T003,ACC101,DEBIT,20000",
  "T004,ACC103,CREDIT,15000",
  "T005,ACC102,DEBIT,45000",
  "T006,ACC101,DEBIT,10000",
  "T007,ACC103,DEBIT,18000",
  "T008,ACC104,DEBIT,25000",
  "T009,ACC104,CREDIT,10000",
  "T010,ACC102,CREDIT,8000"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Map each record to (AccountNo, +Amount) for CREDIT and (AccountNo, -Amount) for DEBIT
- Use reduceByKey() to calculate net balance per account
- Use mapValues() to label 'Overdrawn' if balance < 0, else 'Healthy'
- Filter and count overdrawn accounts — print their account numbers

Pharma | Medium

Q13 Sun Pharma tracks drug inventory across warehouses: 'WarehouseID,DrugName,StockUnits'. Find drugs available in ALL 3 warehouses (universal stock) vs drugs stocked in only ONE warehouse (exclusive stock).

RDD Operations: `map()` | `groupByKey()` | `mapValues()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)
```

```
data = [
  "WH1,Metformin,500", "WH1,Atorvastatin,300",
  "WH1,Aspirin,800",   "WH1,Paracetamol,1000",
  "WH2,Metformin,400", "WH2,Ibuprofen,250",
  "WH2,Aspirin,600",   "WH2,Amoxicillin,350",
  "WH3,Metformin,700", "WH3,Aspirin,450",
  "WH3,Paracetamol,900","WH3,Ciprofloxacin,200"
]
total_warehouses = 3
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Convert to (DrugName, WarehouseID) pairs using map()
- Group by DrugName using groupByKey() to get list of warehouses per drug
- Filter drugs where len(set(warehouses)) == 3 — available in all warehouses
- Filter drugs where len(set(warehouses)) == 1 — exclusive to one warehouse

Telecom | Hard

Q14 Airtel recharge history: 'PhoneNo,Plan,AmountRs,Date'. Using combineByKey(), compute for each customer: count of recharges, total amount spent, and average recharge value. Flag frequent rechargers (more than 3 times).

RDD Operations: `map()` | `combineByKey()` | `mapValues()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "9876543210,Prepaid,199,2024-01-01",
  "9123456789,Postpaid,599,2024-01-02",
  "9876543210,Prepaid,299,2024-01-05",
  "9988776655,Prepaid,149,2024-01-06",
  "9123456789,Postpaid,599,2024-01-10",
  "9876543210,Prepaid,399,2024-01-12",
  "9988776655,Prepaid,199,2024-01-14",
  "9876543210,Postpaid,999,2024-01-15",
  "9111222333,Prepaid,99,2024-01-16",
  "9123456789,Prepaid,249,2024-01-18"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Create (PhoneNo, Amount) pairs using map()
- createCombiner : lambda amt: (1, amt)
- mergeValue : lambda acc, amt: (acc[0]+1, acc[1]+amt)
- mergeCombiners : lambda a, b: (a[0]+b[0], a[1]+b[1])
- Compute avg = total/count in a final mapValues() step
- Filter phone numbers where recharge count > 3

Logistics | Medium

Q15 BlueDart shipment data: 'ShipmentID,OriginCity,DestCity,Status'. Count shipments per route (OriginCity -> DestCity). Find the top 5 busiest routes and routes with only 1 shipment.

RDD Operations: `map()` | `reduceByKey()` | `top()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "SHP001,Mumbai,Delhi,Delivered",
  "SHP002,Delhi,Bangalore,InTransit",
  "SHP003,Mumbai,Delhi,InTransit",
  "SHP004,Chennai,Mumbai,Delivered",
  "SHP005,Delhi,Bangalore,Delivered",
  "SHP006,Hyderabad,Mumbai,Pending",
  "SHP007,Mumbai,Delhi,Delivered",
  "SHP008,Chennai,Mumbai,InTransit",
  "SHP009,Delhi,Bangalore,Pending",
  "SHP010,Kolkata,Delhi,Delivered",
  "SHP011,Hyderabad,Bangalore,InTransit",
  "SHP012,Mumbai,Chennai,Delivered"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Create ((OriginCity, DestCity), 1) pairs using map()
- Aggregate route counts using reduceByKey(lambda a,b: a+b)
- Find top 5 busiest routes using top(5, key=lambda x: x[1])
- Filter routes with exactly 1 shipment — rare / low-volume routes

SECTION 3: RDD Actions

Healthcare | Easy

Q16 Manipal Hospital patient vitals: 'PatientID,BP_Systolic,BP_Diastolic,HeartRate'. Using RDD Actions, find min/max heart rate, average BP_Systolic, and count of critical patients (HR > 120 OR BP_Systolic > 140).

RDD Operations: `map()` | `reduce()` | `min()` | `max()` | `filter()` | `count()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "P001,120,80,75",  "P002,145,92,130",
  "P003,110,70,68",  "P004,160,100,125",
  "P005,130,85,88",  "P006,115,75,72",
  "P007,175,110,135","P008,118,78,65",
  "P009,140,90,95",  "P010,122,82,110"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Extract HeartRate using map() — apply min() and max() actions
- Extract BP_Systolic values and use reduce() to sum, then divide by count()
- Filter critical patients: HeartRate > 120 OR BP_Systolic > 140
- Use count() on filtered RDD to get number of critical patients

eCommerce | Medium

Q17 Myntra product data: 'ProductID,Rating,ReviewCount'. Use RDD Actions to find total products, average rating, count of products rated >= 4.0, and the most-reviewed product.

RDD Operations: `aggregate()` | `filter()` | `reduce()` | `count()` | `top()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "PRD001,4.5,1250", "PRD002,3.8,890",
  "PRD003,4.2,3400", "PRD004,2.9,450",
  "PRD005,4.7,5600", "PRD006,3.5,720",
  "PRD007,4.1,1800", "PRD008,4.8,9200",
  "PRD009,3.2,300",  "PRD010,4.4,2100"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Use count() for total products
- Use aggregate(zeroValue=(0.0,0), seqOp=..., combOp=...) to compute sum & count of ratings in one pass
- Calculate average = sum / count from aggregate result
- Filter products with Rating >= 4.0 and count them
- Use top(1, key=lambda x: int(x.split(',')[2])) to find most-reviewed product

Retail | Easy

Q18 DMart daily sales data: 'StoreID,Category,SalesRs'. Preview records, count total entries, and save filtered high-value sales (Sales > 10000) to an output path for audit purposes.

RDD Operations: `first()` | `take()` | `count()` | `saveAsTextFile()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "ST01,Groceries,8500",
  "ST02,Electronics,25000",
  "ST01,Clothing,4200",
  "ST03,Groceries,15000",
  "ST02,Groceries,9800",
  "ST03,Electronics,32000",
  "ST01,Electronics,11000",
  "ST04,Clothing,6700",
  "ST04,Groceries,18500",
  "ST02,Clothing,3400"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Use first() to inspect schema and first record
- Use take(5) to preview the first 5 records
- Use count() to get total number of daily records
- Filter records where Sales > 10000 using filter()
- Save filtered RDD using saveAsTextFile('/tmp/dmart_high_sales')

Manufacturing | Hard

Q19 Mahindra quality control data: 'ProductLine,DefectType,DefectCount,ShiftID'. Use fold() for total defects, foreach() to log critical types, aggregate() for defect stats, and countByValue() on DefectType.

RDD Operations: `fold()` | `foreach()` | `aggregate()` | `countByValue()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "Engine,Critical,5,Morning",
  "Body,Minor,12,Evening",
  "Engine,Critical,3,Night",
  "Chassis,Major,7,Morning",
  "Body,Critical,2,Morning",
  "Engine,Minor,15,Evening",
  "Chassis,Critical,4,Night",
  "Body,Major,9,Evening",
  "Engine,Major,6,Morning",
  "Chassis,Minor,11,Night"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Extract DefectCount using map() — use fold(0, lambda a,b: a+b) to get grand total
- Use foreach() to print records where DefectType == 'Critical'
- Use aggregate((0,0), seqOp, combOp) to compute (total_defects, max_defects) in one pass
- Extract only DefectType column using map() then apply countByValue()
- Print frequency of each defect type as a sorted dictionary

Pharma | Medium

Q20 Cipla clinical trial data: 'TrialID,DrugName,PatientGroup,SuccessRatePct'. Use lookup() to retrieve all trials for a specific drug, countByKey() for trials per drug, and collectAsMap() for a quick summary dictionary.

RDD Operations: `map()` | `lookup()` | `countByKey()` | `collectAsMap()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "TR001,Metformin,GroupA,78",
  "TR002,Atorvastatin,GroupB,85",
  "TR003,Metformin,GroupC,72",
  "TR004,Aspirin,GroupA,91",
  "TR005,Metformin,GroupB,80",
  "TR006,Atorvastatin,GroupC,88",
  "TR007,Aspirin,GroupB,76",
  "TR008,Ciprofloxacin,GroupA,83",
  "TR009,Aspirin,GroupC,94",
  "TR010,Ciprofloxacin,GroupB,79"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Convert to (DrugName, (TrialID, SuccessRate)) pairs using map()
- Use lookup('Metformin') to retrieve all trial records for Metformin

- Use countByKey() to count number of trials per drug — returns a dict
- Use collectAsMap() to build a drug -> list_of_trials dictionary in driver

SECTION 4: Advanced & Mixed Operations

Media/OTT | Hard

Q21 Netflix India viewing history: 'UserID,ShowID,EpisodeNo,WatchedOn'. Using mapPartitions(), compute binge-watch scores (user watched 3+ episodes of same show in one day). Repartition first for better data co-location.

RDD Operations: `repartition()` | `mapPartitions()` | `reduceByKey()` | `sortBy()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "U01,SHOW1,1,2024-01-10", "U01,SHOW1,2,2024-01-10",
  "U01,SHOW1,3,2024-01-10", "U02,SHOW2,1,2024-01-10",
  "U02,SHOW2,2,2024-01-10", "U03,SHOW1,1,2024-01-11",
  "U01,SHOW2,1,2024-01-11", "U04,SHOW3,1,2024-01-10",
  "U04,SHOW3,2,2024-01-10", "U04,SHOW3,3,2024-01-10",
  "U04,SHOW3,4,2024-01-10", "U05,SHOW1,1,2024-01-10",
  "U05,SHOW1,2,2024-01-10", "U05,SHOW1,3,2024-01-10",
  "U05,SHOW1,4,2024-01-10", "U05,SHOW1,5,2024-01-10"
]
rdd = sc.parallelize(data, 4)
```

Tasks to Perform:

- Repartition to 4 partitions using repartition(4)
- Use mapPartitions() — within each partition, group by (UserID, ShowID, Date)
- Count episodes per (UserID, ShowID, Date) group — flag if count >= 3 as binge session
- Emit (ShowID, binge_score) and aggregate with reduceByKey()
- Sort descending by binge score to find most binge-worthy shows

Banking | Hard

Q22 Kotak Bank fraud detection: 'TxnID,AccountNo,AmountRs,City,Date'. Flag accounts as 'Suspicious' if they have more than 2 transactions > Rs.50,000 on the same date. Use zipWithIndex() to add sequence numbers before processing.

RDD Operations: `zipWithIndex()` | `groupByKey()` | `mapValues()` | `filter()` | `coalesce()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "T001,ACC001,75000,Mumbai,2024-01-15",
  "T002,ACC002,30000,Delhi,2024-01-15",
  "T003,ACC001,80000,Pune,2024-01-15",
  "T004,ACC003,60000,Mumbai,2024-01-15",
  "T005,ACC001,90000,Mumbai,2024-01-15",
  "T006,ACC002,55000,Delhi,2024-01-15",
  "T007,ACC003,70000,Mumbai,2024-01-15",
```

```
  "T008,ACC003,85000,Delhi,2024-01-15",
  "T009,ACC004,45000,Chennai,2024-01-15",
  "T010,ACC002,62000,Hyderabad,2024-01-15"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Use zipWithIndex() to assign a sequence number to every transaction record
- Parse and create ((AccountNo, Date), Amount) key-value pairs
- Group by (AccountNo, Date) using groupByKey()
- In mapValues(), count transactions where Amount > 50000 — flag if count > 2
- Filter and collect flagged (Suspicious) accounts; save with coalesce(1)

Education | Medium

Q23

UpGrad course completion data: 'LearnerID,CourseID,CompletionPct,CertEarned(Yes/No)'. Use sample() to analyze a 40% random subset, compute completion rate and certification rate, then compare with the full dataset.

RDD Operations: `sample()` | `map()` | `reduce()` | `count()` | `filter()`

```
# Sample Dataset  (use sc.parallelize() to load)

data = [
  "L001,C01,100,Yes", "L002,C01,85,No",
  "L003,C02,60,No",   "L004,C02,100,Yes",
  "L005,C03,45,No",   "L006,C01,100,Yes",
  "L007,C03,90,Yes",  "L008,C02,30,No",
  "L009,C01,100,Yes", "L010,C03,75,No",
  "L011,C02,100,Yes", "L012,C01,55,No",
  "L013,C03,100,Yes", "L014,C02,80,No",
  "L015,C01,100,Yes", "L016,C03,40,No"
]
rdd = sc.parallelize(data)
```

Tasks to Perform:

- Use sample(withReplacement=False, fraction=0.4, seed=42) to get sample subset
- Compute avg CompletionPct on sample: use reduce() to sum then divide by count()
- Compute certification rate on sample: filter CertEarned=='Yes' / total count
- Repeat both calculations on the full RDD for comparison
- Print side-by-side: Sample Stats vs Full Dataset Stats

Logistics | Hard

Q24

FedEx India has two RDDs — Shipments ('ShipmentID,RouteKey,WeightKg') and Pricing ('RouteKey,PricePerKg'). Demonstrate cartesian() risk on a small sample, then use the correct join() approach to compute actual shipment cost per record.

RDD Operations: `cartesian()` | `join()` | `map()` | `reduceByKey()`

```
# Sample Dataset  (use sc.parallelize() to load)

shipments_data = [
  "SH001,MUM_DEL,15.0", "SH002,DEL_BLR,8.5",
```

```
  "SH003,MUM_DEL,22.0", "SH004,HYD_MUM,12.0",
  "SH005,DEL_BLR,6.0",  "SH006,CHN_DEL,18.5",
  "SH007,MUM_DEL,9.0",  "SH008,HYD_MUM,25.0"
]
pricing_data = [
  "MUM_DEL,45.0",
  "DEL_BLR,55.0",
  "HYD_MUM,40.0",
  "CHN_DEL,50.0"
]
shipments_rdd = sc.parallelize(shipments_data)
pricing_rdd   = sc.parallelize(pricing_data)
```

Tasks to Perform:

- First: demonstrate cartesian() on first 3 shipments x all pricing rows — observe the explosion (3x4=12 rows)
- Explain why cartesian() is dangerous on large data (M x N rows)
- Create (RouteKey, (ShipmentID, WeightKg)) from shipments using map()
- Join with pricing_rdd on RouteKey — compute cost = WeightKg * PricePerKg
- Use reduceByKey() to compute total revenue per route

Healthcare | Hard

Q25 AIIMS multi-source pipeline — 3 RDDs: Patients ('PatientID,DoctorID,Diagnosis'), Doctors ('DoctorID,Name,Specialization'), Billing ('PatientID,TotalBillRs'). Build a full patient report and find the top 3 highest-billing specializations.

RDD Operations: `join()` | `leftOuterJoin()` | `map()` | `reduceByKey()` | `top()`

```
# Sample Dataset  (use sc.parallelize() to load)

patients_data = [
  "PAT01,DOC01,Diabetes",  "PAT02,DOC02,Fracture",
  "PAT03,DOC01,Hypertension","PAT04,DOC03,Cancer",
  "PAT05,DOC02,Appendicitis","PAT06,DOC03,Cancer",
  "PAT07,DOC04,Asthma",    "PAT08,DOC01,Diabetes"
]
doctors_data = [
  "DOC01,Dr. Mehta,Endocrinology",
  "DOC02,Dr. Sharma,Orthopedics",
  "DOC03,Dr. Reddy,Oncology",
  "DOC04,Dr. Iyer,Pulmonology"
]
billing_data = [
  "PAT01,15000", "PAT02,45000",
  "PAT03,12000", "PAT04,180000",
  "PAT05,35000", "PAT06,200000",
  "PAT07,8000",  "PAT08,18000"
]
patients_rdd = sc.parallelize(patients_data)
doctors_rdd  = sc.parallelize(doctors_data)
billing_rdd  = sc.parallelize(billing_data)
```

Tasks to Perform:

- Step 1 — Convert patients_rdd to (DoctorID, (PatientID, Diagnosis)) and join with doctors_rdd
- Step 2 — From joined result, extract (PatientID, Specialization) pairs
- Step 3 — Convert billing_rdd to (PatientID, TotalBill) and join with Step 2 result
- Step 4 — Create (Specialization, TotalBill) pairs and aggregate with reduceByKey()

- Step 5 — Use top(3, key=lambda x: x[1]) to find highest-billing specializations
- Step 6 — Discuss why caching intermediate RDDs (patient+doctor join) improves performance here

Sateesh Says:
Remember: Transformations are LAZY — they build the recipe. Actions are EAGER — they cook the data! Always .cache() RDDs you reuse, like saving your Swiggy order as a favourite — no need to search again!

— End of Assignment —

LevelUp Cloud Trainings | Hyderabad | levelupedu.net